OS ORANGE PROBLEM(UNIT-4)

NAME: Shreyas Mohanty

SRN: PES2UG24CS491

SCREENSHOT 1A:

```
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$
```

SCREEENSHOT 1B:

```
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$
```

SCREENSHOT 2A:

```
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$
```

SCREENSHOT 3A:

```
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$ echo "hello" > file1.txt
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$ echo "world" > file2.txt
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$ ./pes add file1.txt file2.txt
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_tree.c
  untracked:  test_objects.c
  untracked:  Makefile
  untracked:  pes.c
  untracked:  tree.h
  untracked:  tree.c
  untracked:  object.c
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  README.md
  untracked:  commit.h
  untracked:  test_sequence.sh
  untracked:  index.c
  untracked:  index.h
  untracked:  commit.c

s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$
```

SCREENSHOT 3B:

```
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776259359 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776259367 6 file2.txt
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$
```

SCREENSHOT 4A:

```
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$ ./pes log
commit 7bf03f6099f11686711161e3f3aa1cfa8377fee98f8652887970121d5e80a9d5
Author: Shreyas Mohanty <PES2UG24CS491>
Date:   1776261200

    Add farewellinsta !

commit 5739a3ae253b1ebb6ba5834b6914050d564a25b407a0cd87a936f597c60285c0
Author: Shreyas Mohanty <PES2UG24CS491>
Date:   1776261178

    Add worldellinsta !

commit 4bbee80b6838f7d66166345e94d3f3f262568d255b222a7be3cc691c92f5bb54
Author: Shreyas Mohanty <PES2UG24CS491>
Date:   1776261142

    Initial committial commitEA

s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$
```

SCREENSHOT 4B:

```
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/4b/bee80b6838f7d66166345e94d3f3f262568d255b222a7be3cc691c92f5bb54
.pes/objects/57/39a3ae253b1ebb6ba5834b6914050d564a25b407a0cd87a936f597c60285c0
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7b/f03f6099f11686711161e3f3aa1cfa8377fee98f8652887970121d5e80a9d5
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$
```

SCREENSHOT 4C:

```
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$ cat .pes/refs/heads/main
7bf03f6099f11686711161e3f3aa1cfa8377fee98f8652887970121d5e80a9d5
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$
```

Final integration test ss:

```
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: b6c93163862b... Initial commit

Log after first commit:
commit b6c93163862bb252385f06a151b48292d886d97b1c64682a490bebf45ccd42a3
Author: Shreyas Mohanty <PES2UG24CS491>
Date:   1776266361

    Initial commit


--- Second Commit ---
Committed: 97a9f051415b... Update file.txt

--- Third Commit ---
Committed: 1ef2983a0ec8... Add farewell
```

```
--- Full History ---
commit 1ef2983a0ec858fd7f94358a17c823c244f44cc694fadbc933abdbe09eebc8fb
Author: Shreyas Mohanty <PES2UG24CS491>
Date:   1776266362

    Add farewellinsta !

commit 97a9f051415ba9cd71a75a27ba0ee44b4ce1a49a5cf18a771c3eb0cce9e9099e
Author: Shreyas Mohanty <PES2UG24CS491>
Date:   1776266362

    Update file.txtta !

commit b6c93163862bb252385f06a151b48292d886d97b1c64682a490bebf45ccd42a3
Author: Shreyas Mohanty <PES2UG24CS491>
Date:   1776266361

    Initial committial commitEA


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
1ef2983a0ec858fd7f94358a17c823c244f44cc694fadbc933abdbe09eebc8fb

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/1e/f2983a0ec858fd7f94358a17c823c244f44cc694fadbc933abdbe09eebc8fb
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/97/a9f051415ba9cd71a75a27ba0ee44b4ce1a49a5cf18a771c3eb0cce9e9099e
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/b6/c93163862bb252385f06a151b48292d886d97b1c64682a490bebf45ccd42a3
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
s_monty@shreyas-VirtualBox:~/Documents/OS_PES/OS-Orange(u4)/PES2UG24CS491-pes-vcs$
```

PHASE 5 (BRANCHING & CHECKOUT)

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Answer:
To implement pes checkout <branch>, first check if .pes/refs/heads/<branch> exists.
- If it exists, read the commit hash from that file.

- Update .pes/HEAD to: ref: refs/heads/<branch>.

After changing HEAD, update the working directory so it matches that commit's tree:
1. Read the commit object.
2. Read its root tree.
3. Recreate files/folders from that tree in the working directory.
4. Remove tracked files that are not part of the target branch snapshot.

Also update .pes/index so it matches the checked-out snapshot.

Why it is complex:
- You must not lose user changes.
- You must detect conflicts (files changed locally but different in target branch).
- You need careful file deletion/creation order for nested folders.
- Checkout is not just one file update; it's metadata + full workspace rewrite.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Answer:
Use a 3-way comparison:

For each tracked file:
1. Read current index entry (this is what user last staged).
2. Read actual file from working directory and check:
   - mtime/size quick check, or
   - recompute blob hash for exact check.
3. Read target branch version of that same path from target tree (via object store).

Conflict rule:
- If working directory file is different from current index (so user has local uncommitted change),
AND
- target branch version is also different from current version, then checkout must stop with an error (to avoid overwriting local work).

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Answer:
In detached HEAD, new commits are created normally, but no branch name moves forward.
So commits exist, but they are “floating” and can be lost later if nothing points to them.

How to recover:
- Immediately create a branch pointing to that commit:
  pes branch rescue <commit-hash> (or equivalent logic)
- Or switch to a branch and then attach that commit by creating a branch at that hash.

PHASE 6 (GARBAGE COLLECTION & SPACE RECLAMATION)

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Answer:
Use a classic mark-and-sweep algorithm.

Step 1: Mark reachable
- Start from all branch heads in .pes/refs/heads/*.
- For each commit:
  - mark commit hash as reachable,
  - mark its tree as reachable,
  - follow parent commit and continue.
- For each tree:
  - mark subtree hashes and blob hashes reachable recursively.

Store reachable hashes in a Hash Set (fast O(1) average insert/lookup).

Step 2: Sweep

- Walk all files under .pes/objects.
- If object hash is not in reachable set, delete it.

Why hash set:
- Very fast existence checks.
- Handles large repos efficiently.

Estimate for 100,000 commits and 50 branches:
- If branches mostly share history, visited commits are still around ~100,000 (not 5,000,000).
- Each commit references one tree, and trees reference blobs/subtrees.
- Rough total object visits may be a few hundred thousand to a few million depending on project size and file churn.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Answer:
The danger is commit creation happens in steps. If GC runs at the same time, it may think new objects are unreachable before refs are updated.

Race example:
1. Commit process writes new tree/blob/commit objects.
2. Branch ref is not updated yet (still points to old commit).
3. GC scans refs, does not see new objects as reachable.
4. GC deletes them.

5. Commit process now updates ref to a commit whose objects are gone -> broken repository.

How real Git avoids this:
- Uses safety rules and grace periods (recent objects are protected).
- Uses locking/coordination around refs and object updates.
- Uses packfiles and reachability logic carefully.
- Usually avoids aggressive deletion of very new loose objects.